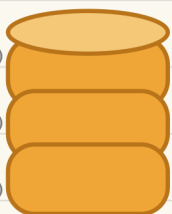


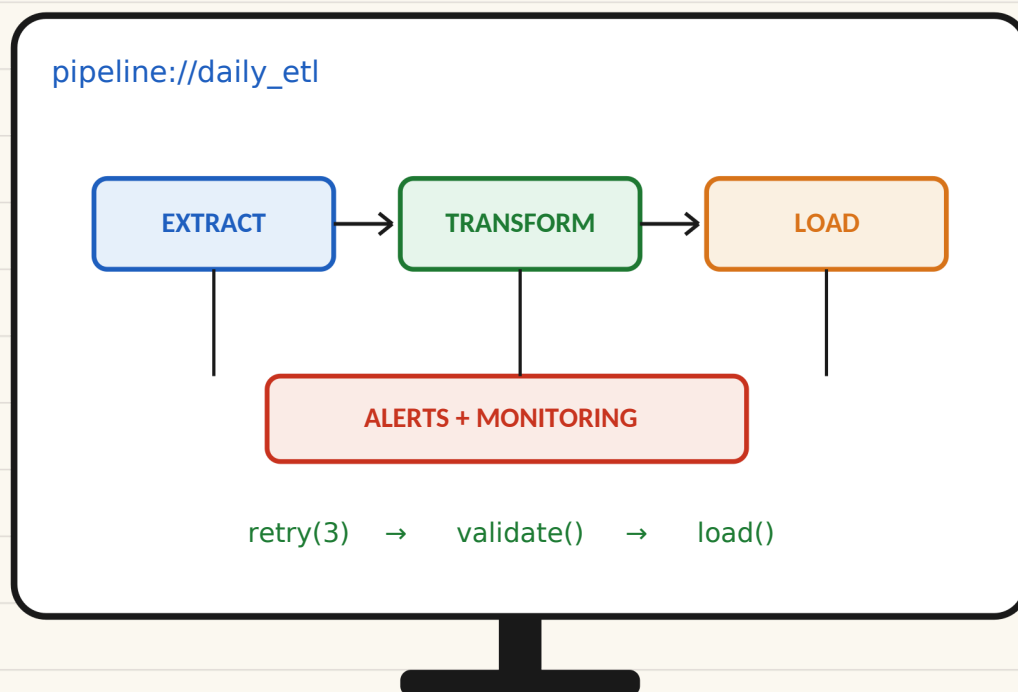
Data Engineer

INTERVIEW Q & A

≈ (14 Real-World Scenarios with Answers) ≈



DATA OPS



DAILY ISSUES • BEGINNER TO ADVANCED



RQ_Data_Talks



RQanesh0203

SWIPE TO NEXT →



Source notes by Nishant Lakhapati · reformatted as an interview Q&A guide

Handling ETL Job Failures

Q1 • Why does failure handling matter?

Failures are guaranteed — bad files, network blips, schema drift. The goal is **data integrity** and a pipeline that **fails loudly** and recovers cleanly.

- **Catch** exceptions per step, not per job
- **Log** timestamp + job id + error type
- **Alert** the right people fast

Q2 • Error handling & alerts

Try / catch around each ETL step so one bad record cannot kill the whole run. **Error logs** carry job id, timestamp and error type — to files, a DB or a monitoring system.

Alerts go out by email, SMS or chat with the job, the failure and the logs. Track live status in **Azure Monitor**, **CloudWatch** or a custom dashboard.

Q3 • Validation, retries & fallbacks

Layer	What you do	Example
Validate	Pre + post checks on every load	Schema, data-type and range checks
Profile	Watch incoming data for anomalies	Sudden null spike, new category
Retry	Auto-retry transient errors only	Exponential backoff, max 3-5 tries
Escalate	Manual review for persistent failures	Engineer decides the fix
Degrade	Load valid rows, park the bad ones	Partial load + reject table
Roll back	Restore last good load	Backup or Delta time travel

```
for attempt in range(1, MAX_RETRIES + 1):
    try:
        run_step()
        break
    except TransientError as e:
        log.warning("attempt %s failed: %s", attempt, e)
        time.sleep(2 ** attempt)           # exponential backoff
    except PermanentError:
        alert_team(); raise                 # do not retry bad data
```

Tip

Separate **transient** errors (retry) from **permanent** ones (alert + fix). Retrying a schema error just wastes compute and hides the real problem.

ETL Failures — Prevention

Q4 • Document, RCA & test

Practice	Why it earns its place
Failure log / knowledge base	Cause + fix recorded, so the same page is never paged twice
Root Cause Analysis	Fix the cause, not the symptom — then add a check to catch it earlier
Unit tests	Each transform tested with small known inputs
Integration tests	All components work together before release
Git for ETL code + config	Roll back to the last stable version in minutes
Change management	Structured, reviewed updates cause fewer surprise outages

Q5 • Maintenance & optimisation

- **Performance monitoring** — optimise bottlenecks before they turn into failures
- **Scheduled maintenance** — check resource limits, source changes and configs on a routine
- **Backups** of the previous successful load — the fastest rollback you can have

Q6 • The failure-handling checklist

Errors caught, classified and logged with context
Alert fires with job, failure type and logs attached
Transient errors retry; permanent errors stop and escalate
Bad rows quarantined — good rows still load
Backup available; rollback tested
RCA written, prevention added

★ Interview Tip

Answer in this order: **detect** → **contain** → **recover** → **prevent**. Then add one real story — the failure, how you found it, what you changed. That beats a memorised list.

Pipeline Running Slower

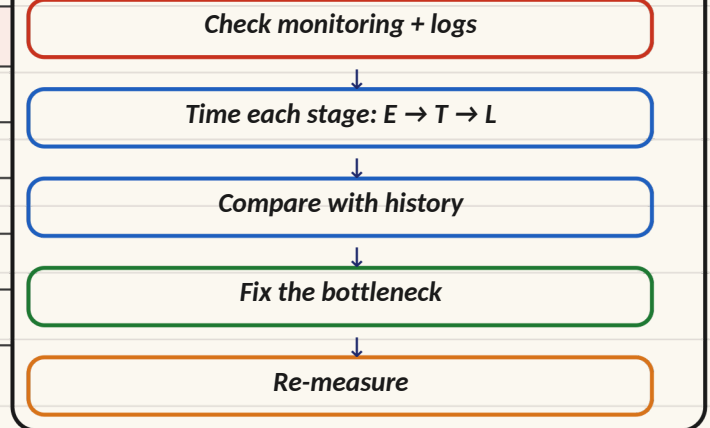
Q7 • First move: measure, don't guess

Open monitoring (Azure Monitor, CloudWatch or a dashboard) and read the logs. Watch **execution time**, CPU, memory, I/O and **throughput**. Then split the pipeline into Extract → Transform → Load and find the **one stage** that grew.

Q8 • Where is the bottleneck?

Check	Look for
Data volume	Sudden growth vs last runs
Resources	Under-provisioned CPU / RAM
Data quality	Corrupt or inconsistent rows
Network	Latency to remote sources
Recent changes	New code or config release

HOW IT FLOWS



Q9 • Fix it: optimisation levers

Lever	What to do
Scale	Scale up (bigger nodes) or scale out (more nodes)
Transform logic	Remove needless joins, calculations and type casts; prefer built-in functions
Parallelism	Split work across threads/workers; partition data evenly
Caching	Cache data read repeatedly; store expensive results in Delta / temp tables
Batch vs stream	Smaller, more frequent batches cut latency; go streaming if real-time is needed
Transfers	Compress data and use efficient protocols between components

```
# spot the slow stage in Spark
df.explain("formatted")
df.rdd.getNumPartitions()
```

```
# is there an unexpected Exchange (shuffle)?
# too few = idle cores, too many = overhead
```

Slow Pipeline — Verify & Iterate

Q10 • Test, benchmark, repeat

- **Isolate** — run in a test environment to see if the slowdown is the data or the code
- **Benchmark** against historical run times to find *when* it started slowing
- **Review config** of ETL tools and databases; check recent commits with version control
- **Change one thing at a time** and measure each change

Q11 • Slow-pipeline triage table

Symptom	Likely cause	First action
One stage suddenly 3x slower	Data volume or skew	Compare row counts + partition sizes
Everything slower, same data	Resource contention / config	Check cluster metrics + recent config
Slow only at load step	Target DB or network	Check write throughput + latency
Slow after a release	Code change	Diff the commit; roll back to confirm

Q12 • Collaborate & iterate

Bring in the owners of upstream sources and downstream consumers — the cause is often outside your code. Ship improvements iteratively and keep a before/after runtime for each.

★ Interview Tip

Diagnosis before tuning. Say: "I'd find the slowest stage, then check whether it's data growth, resources, or my own transform logic." Never list configs first.

💡 Tip

Keep runtime history per job. A benchmark you can compare against turns "it feels slow" into "it slowed on the 14th after the schema change".

Data Quality at Scale

Q13 • The six dimensions of data quality

Dimension	Question it answers
Accuracy	Does the data match the real world?
Completeness	Is all required data present?
Consistency	Does it agree across datasets and sources?
Timeliness	Is it up to date when needed?
Uniqueness	Are there duplicate entries?
Validity	Does it follow defined formats and standards?

Q14 • Profile & identify

Profile first: distributions, missing values, outliers. Tools: Talend, Apache Griffin, Informatica.

- Missing / null values
- Inaccurate or outdated entries
- Duplicate records
- Mixed date / currency formats

Q15 • Cleanse

Issue	Fix
Missing	Mean/mode, interpolation, domain rules
Inaccurate	Reference data + business rules
Duplicates	Dedup on the business key
Formats	Standardise dates, case, units

```
from pyspark.sql import functions as F
# null count per column — the fastest profiling step
df.select([F.count(F.when(F.col(c).isNull(), c)).alias(c) for c in df.columns]).show()
```

Data Quality – Governance

Q16 • Validate & automate

Turn business requirements into **validation rules** — range checks, format checks, consistency checks — and run them automatically on every load. Build the checks **into the ETL**, not after it.

- Dedicated tools: **Talend DQ**, **Trifacta**, **Informatica DQ**
- Rules run at every stage of the data lifecycle
- Failing rows go to a quarantine table with a reason

Q17 • Governance, monitoring & people

Area	Practice
Stewardship	Named data owners per domain
Framework	Written policies, procedures, responsibilities
Monitoring	Quality metrics on dashboards; trend over time
Feedback loops	Learn from every issue; improve the process
Training	Teach users why clean entry matters
Iterate	Review rules as sources and business needs change

Q18 • Quality checks you can say out loud

- ✗ Row count vs source (reconciliation)
- ✗ Primary key unique + not null
- ✗ Accepted values and range checks on amounts
- ✗ Referential check with left_anti against the dimension
- ✗ Freshness: max(event_ts) vs SLA

★ Interview Tip

"Quarantine, don't drop." Write failing rows to a rejects table with a reason column. Interviewers hear the senior answer: nothing silently disappears.

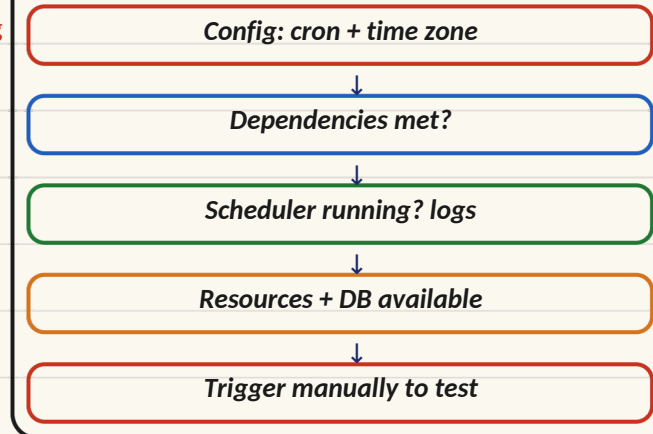
Scheduled Job Didn't Trigger

Q19 • Where to look first

Work from the outside in. Most missed triggers are a **config** problem, not a code problem.

- **Schedule** — cron expression and start time
- **Time zone** — UTC vs local is the #1 culprit
- **Dependencies** — did the upstream job finish?
- **Recent changes** — code, config, environment

HOW IT FLOWS



Q20 • Checklist by layer

Layer	Check	Tools
Job config	Schedule, frequency, start time	Cron, Airflow, ADF
Scheduler	Is it running? Restart if needed; read its logs	Scheduler UI + logs
Resources	CPU, memory, disk; DB reachable	Monitoring dashboards
Time zone	Job zone = expected zone	Scheduler settings
Changes	Recent commits or env changes; roll back to isolate	Git history

```
# Airflow: a classic trap — start_date + schedule
DAG("daily_sales",
    start_date=datetime(2026, 9, 1),
    schedule="0 2 * * *",          # 02:00 UTC, not local time
    catchup=False)
```

Missed Trigger — Recover & Prevent

Q21 • Test, then fix

- **Manually trigger** the job if it is safe. If it runs, the problem is the schedule; if it fails, it is the job.
- **Read the docs** for the scheduler — settings are easy to overlook
- **Escalate** to vendor support or community forums when it persists

Q22 • Prevent the next miss

Action	Effect
Alert on failed and missed runs	You hear about it in minutes, not next morning
Re-evaluate frequency vs workload	Avoid peak-hour resource contention
Document cause and resolution	Faster diagnosis next time
SLA-based sensors	Alarm if a job has not started by its deadline

Q23 • Common causes

- Wrong time zone or cron expression
- Upstream dependency not finished
- Scheduler down or stuck
- Resource / DB connection unavailable
- Recent code or config change

★ Interview Tip

Most people forget the silent failure. Say: "I alert on a job that *didn't start*, not only one that failed." A missing run produces no error to catch.

Spark Memory Troubleshooting

Q24 • How Spark uses memory

- **Execution** — shuffles, joins, aggregations
- **Storage** — cached RDDs / DataFrames
- **Unified model** — the two borrow from each other dynamically

Symptoms: **slow stages**, **OutOfMemoryError**, retried tasks, crashed apps.

Q25 • Settings to check

Config	Controls
<code>spark.driver.memory</code>	Driver heap
<code>spark.executor.memory</code>	Executor heap
<code>spark.dynamicAllocation.*</code>	Executor scaling
<code>spark.serializer</code>	Kryo for speed
<code>spark.sql.shuffle.partitions</code>	Post-shuffle parallelism

Q26 • Diagnose with the Spark UI + logs

Where	Look for
Stages tab	Long stages, high shuffle, spill to disk
Tasks	Failed / retried tasks; max median duration
Executor logs	<code>java.lang.OutOfMemoryError</code> , killed containers
Memory metrics	Usage per executor via UI or API
Heap dump	Last resort — find leaks and huge objects

Q27 • Reduce the data and the shuffles

- **Filter / aggregate early** so less data reaches each stage
- **Partition size 128-256 MB** — `repartition()` or `coalesce()` to balance
- **Avoid wide transforms** like `groupByKey`; use broadcast for small lookups
- **Kryo serialisation** — smaller and faster than Java serialisation

Memory — Tuning Playbook

Q28 • GC, caching & parallelism

Area	Action
Garbage collection	Use G1GC (-XX:+UseG1GC) on large heaps; enable GC logs
Caching	Cache only what is reused; unpersist() when done
Storage level	MEMORY_AND_DISK when RAM is tight
Parallelism	Tune spark.default.parallelism + shuffle partitions to cluster size
Skew	Salting or a different join strategy for hot keys
Code	Remove needless copies, giant intermediates, excess caching

```
spark.conf.set("spark.sql.shuffle.partitions", 400)
spark.conf.set("spark.serializer", "org.apache.spark.serializer.KryoSerializer")
df.persist(StorageLevel.MEMORY_AND_DISK)      # ... use ...
df.unpersist()                                # free it
```

Q29 • Error → cause → fix

Real cause Error		Fix
Driver OOM	collect() / toPandas() on big data	Aggregate first; write to storage
Executor OOM	Skewed or oversized partitions	More partitions; fix skew
Container killed	Off-heap overhead (Python workers)	Raise memoryOverhead
Slow + spilling	Partitions too big for memory	Repartition; raise executor memory

★ Interview Tip

Start with **partitions size and skew** before adding memory. Throwing RAM at an unbalanced job hides the problem and raises the bill.

◆ Tip

PySpark OOM is often **overhead, not heap**: Python workers live outside the JVM heap. Raise memoryOverhead before executor memory.

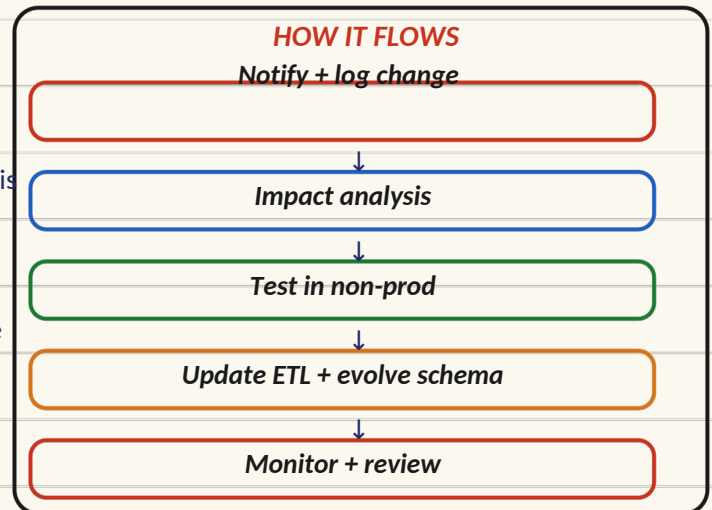
Schema Changes in Source Systems

Q30 • Why schema change breaks pipelines

A renamed column or new type causes **job failures**, **silent bad data** and inconsistent downstream reports. Treat schema change as a **managed event**, not a surprise.

Q31 • Process

- **Change notification** — alerts, meetings or doc updates from the source team
- **Schema versioning** — metadata repo or Git, so rollback is possible
- **Impact analysis** — which pipelines, models and reports break?
- **Stakeholders** — analysts and business users confirm the effect



Q32 • Handling strategies

Strategy	How
Backward compatible	ETL accepts old + new schema until everyone migrates
Schema evolution	Delta Lake / Iceberg add columns automatically
Config-driven mapping	Dynamic column mapping instead of hard-coded names
Explicit schema on read	Fail loudly when the source changes, never guess

```
(df.write.format("delta")
  .mode("append")
  .option("mergeSchema", "true")      #allow new columns
  .save(path))
```

Schema Changes — Test & Communicate

Q33 • Test and validate

- Use a **separate test environment** and run the ETL on the new schema before production
- **Data quality checks** confirm formats and values still conform after the change
- Update ETL logic: column names, data types, transformation rules

Q34 • Monitor, communicate, document

Area	Action
Monitoring	Warnings / errors on schema mismatch
Alerts	Notify the team the moment a job fails on schema
Communication	Tell engineering, analytics and BI what changed
Training	Show affected users how to adapt
Documentation	Update data dictionaries, workflows, user guides
Metadata / lineage	Track schema details and how data flows

Q35 • Post-implementation review

After rollout, review the impact, note what surprised you, and improve the change-management process for the next one.

★ Interview Tip

Ask: "How do you know a source changed before your job fails?" — Answer: schema check at ingestion, explicit schema on read, alert on drift.

◆ Tip

`printSchema()` before every join or write. Half of all "job failed in prod" stories are a silently changed source column type.

Data Partitioning

Q36 • Choose the right strategy

Strategy	Divides by	Best for
Range	Ranges of a column (e.g. dates)	Time series; date-range queries
Hash	Hash of one or more columns	Even spread; avoiding skew
List	Specific values in a column	Categorical data queried often
Composite	Two strategies combined	Specific mixed access patterns

Q37 • Start with access patterns

Ask which columns are **filtered, queried and aggregated** most, and the data's volume, variety and velocity. Partition on what queries filter by.

Target size: 128-256 MB per partition — big enough to cut overhead, small enough to process efficiently.

Q38 • Partitioning in ETL

- Partition as data is loaded
- **Dynamic partitioning:** new partitions appear as data arrives
- **Incremental loads** touch only new or changed partitions

```
(df.write
  .partitionBy("year", "month")      # folders on disk
  .mode("overwrite")
  .parquet("/lake/sales"))
# WHERE year = 2026 AND month = 9 → only that folder is read
```

Partitioning — Performance & Lifecycle

Q39 • Pruning, skew and small files

Concern	Do this
Partition pruning	Filter on partition keys in WHERE so only relevant partitions are scanned
Data skew	Watch partition sizes; salt hot keys or rethink the key
Over-partitioning	Millions of tiny folders hurt metadata and speed
Retention	Archive or delete old partitions on a schedule
Review	Reorganise as usage patterns change

Q40 • Storage and formats

- Data lakes: [Amazon S3](#), [Azure Data Lake](#); engines: [Hive](#), [Spark](#)
- **Parquet / ORC** — columnar formats that make partitioned reads fast
- Document the strategy — rationale, config, caveats — and share it with analysts

Q41 • Partitioning checklist

- ✓ Key matches the most common filter
- ✓ Low cardinality (tens to thousands of values)
- ✓ Partition files land near 128–256 MB
- ✓ Pruning confirmed in the query plan
- ✓ Skew and small-file counts monitored

★ Interview Tip

"Whynotpartitionby customer_id?" — Millions of tiny folders and catastrophic metadata overhead. Partition on a low-cardinality column; bucket or Z-order the high-cardinality key.

Third-Party API Ingestion Failure

Q42 • Read the error

Code	Meaning
400	Bad request / data format
401 / 403	Auth or permission
404	Wrong endpoint
429	Rate limit hit
500+	Provider-side failure

HOW IT FLOWS

Detect: status, latency, volume



Read the error response



Find root cause



Retry with backoff



Fallback + notify

Q43 • Find the root cause

- **API docs** — request format, auth and limits
- **Network** — DNS, firewall, connectivity
- **Provider status page** — outage or maintenance window
- **Credentials** — API key or token expired?

Q44 • Retry logic

```
import time, random, requests
for attempt in range(5):
    r = requests.get(url, headers=h, timeout=30)
    if r.status_code == 200: break
    if r.status_code in (400, 401, 403, 404): raise
    time.sleep(2 ** attempt + random.random())
```

max retry limit

permanent

backoff + jitter

API Failure — Fallback & Recovery

Q45 • Fallbacks and stakeholders

Situation	Action
Short outage	Serve previously cached data
Prolonged outage	Switch to an alternative source
Downstream impact	Alert data engineers + product owners
Ongoing failure	Send status updates and expected delay

Q46 • Make ingestion resilient

Improvement	How
Enhanced logging	Log request payload, timestamp, API error message
Error classification	Transient vs permanent, client-side vs server-side
Respect rate limits	Throttle calls so you are never blocked
Monitoring dashboard Docs	Health of the ingestion at a glance
+ post-incident review	Record what went wrong and the fix

Q47 • Never retry these

✗/font> 400 — fix the request, not the timing
✗/font> 401 / 403 — refresh credentials first
✗/font> 404 — the endpoint is wrong
✗/font> Any error with no maximum retry cap

★ Interview Tip

Say: "I retry transient errors with exponential backoff and jitter, cap the attempts, and never retry a 4xx." That one sentence shows you know the difference.

💡 Tip

Keep the raw API response in a landing zone before parsing. If parsing fails you can replay it — without calling the API again.

Data Consistency Across Stores

Q48 • Map, then standardise

- **Data-flow mapping** — how data moves between stores
- **Find the origin of drift**: extract, transform or load?
- **Standards** for formats, types and naming
- **Data stewards** own quality per system

Q49 • One source of truth

Consolidate into a **data warehouse or lake** so every consumer reads the same numbers. Use robust ETL so data is extracted, transformed and loaded the same way every time.
Fewer copies = fewer disagreements.

Q50 • Keep stores in sync

Technique	When to use it
Change Data Capture (CDC)	Real-time: propagate source changes to targets
Batch synchronisation	Less time-sensitive; periodic reconcile
API-based integration	Immediate consistency, fetch + update across systems
Integration tools	NiFi, Talend, Informatica — workflows + built-in checks

Q51 • Audit and validate

- **Regular audits** — compare key metrics between systems
- **Automated quality checks** at each pipeline stage

```
# reconcile two stores
src = spark.table("crm.orders").agg(F.count("*"), F.sum("amt"))
tgt = spark.table("dwh.orders").agg(F.count("*"), F.sum("amt"))
assert src.collect() == tgt.collect(), "stores diverged"
```

Consistency — Conflicts & Monitoring

Q52 • Resolve conflicts

Strategy	How it works	Trade-off
Last write wins	Newest timestamp prevails	Simple; can lose valid edits
Versioning	Keep record history; revert if needed	More storage; full audit trail
Source-of-truth rule	One system always wins	Clear; needs governance

Q53 • Monitor consistency

- **Real-time dashboards** and alerts for discrepancies
- **Data lineage** — see where a value came from and where it changed

Q54 • People and process

Area	Action
Training	Teach users why consistent entry matters
Documentation	Data entry standards and the impact of inconsistency
Post-incident review	Find the root cause, fix the process
Continuous improvement	Tighten governance and integration each cycle

★ Interview Tip

Ask yourself: "which store is the source of truth?" If you cannot answer that in one word, consistency problems are guaranteed.

◆ Tip

Reconcile **counts** and **sums** per day between source and target. Two numbers catch most silent divergence.

Job Exceeds Its Time Window

Q55 • Diagnose

- **Logs** — where does it spend the time?
- **Execution plan** — shuffles, big joins
- **Data volume** — grown vs previous runs?
- **Resources** — CPU, memory, I/O limits

HOW IT FLOWS

Read logs + metrics



Find the slow stage



Check volume + resources



Optimise + resize



Alert before the limit

Q56 • Fix it

Approach	Details
Add resources	More executors / memory if consistently limited
Tune parallelism	More parallel tasks — without overwhelming the cluster
Simplify logic	Remove needless joins + aggregations; filter early
Cache	Reuse intermediate results instead of recomputing
Modularise	Split into smaller jobs; easier to tune each

Q57 • Incremental processing

Do not reprocess everything. Load only new or changed data.

```
last = spark.sql("SELECT MAX(ts) FROM tgt").first()[0]
new = src.filter(F.col("ts") > last)
# MERGE new INTO tgt ON key
```

Time Window — Scheduling & Follow-up

Q58 • Schedule smartly

Lever	Action
Timing	Move off peak-usage hours
Prioritisation	Critical jobs get resources first
Retry logic	Backoff for transient failures
Alerts	Warn when a job nears its limit, not after it misses

Q59 • Communicate and learn

- Tell stakeholders if it keeps happening; gather their pain points
- **Post-mortem** — root cause + lessons learned
- **Dashboards** — job duration trends catch drift before it becomes an outage

Q60 • Overrun checklist

- Which stage grew? (logs + plan)
- Did data volume change?
- Are resources at their limit?
- Can it be incremental?
- Can the schedule move?
- Is an alert set before the deadline?

★ Interview Tip

Scenario: "the 2-hour job now takes 5 — what changed?" Check data volume first, then skew, then resources, then a recent release. In that order.

Tip

Alert at 80% of the window, not 100%. An early warning gives you time to act.

Pipeline Dependencies

Q61 • Document + orchestrate

- **Dependency map** — upstream sources, downstream consumers
- **Metadata repo** — schemas, refresh frequency, quality rules
- **Orchestrator**: Airflow, Prefect, Luigi, Dagster, NiFi

Q62 • Monitor + protect

- Job time, success / failure rate, quality checks
- **Alerts** when a dependency is affected
- **Failure policy**: retry, skip or halt downstream
- **Version control** + change management

Q63 • Define dependencies as code

```
extract >> validate >> transform >> load
[extract_orders, extract_users] >> join_facts
load >> [refresh_dashboard, notify_team]
```

```
# runs strictly in order
# fan-in: wait for both
# fan-out after load
```

Q64 • Test and improve

Practice	Purpose
Integration tests	Components work together after upstream changes
End-to-end tests	Data flows correctly through every dependency
Dependency graphs	Visualise bottlenecks and single points of failure
Post-mortems	Improve dependency handling after every incident

★ Interview Tip

"Upstream failed at 2 a.m. — what happens downstream?" Answer with your policy: downstream tasks wait, retry on transient errors, alert on persistent ones, and never run on stale or partial data.

◆ Tip

Prefer **data-aware sensors** (wait until the file or partition exists) over fixed time delays. Time-based waits cause both idle time and missed data.

Incorrect Transformation Output

Q65 • Identify and reproduce

- Pin down the **exact anomaly** — value, count or format
- Compare with the spec or a known-good output
- **Re-run** on the same input to confirm it repeats
- Use a **small controlled test set** to isolate the step

HOW IT FLOWS

Compare with expected output



Trace logs to the step



Reproduce with same input



Inspect logic + input data



Fix, test, regress

Q66 • Where bugs usually hide

Area	Typical bug
Calculations	Wrong formula or rounding
Data types	Implicit cast; string vs number
Conditionals	Wrong branch or missing otherwise()
Joins	Non-unique key → fan-out duplicates
Mappings	Source field mapped to the wrong target
Input data	Malformed or changed upstream values

```
# does the join multiply rows?
dim.groupBy("prod_id").count().filter("count > 1").show()
print(fact.count(), fact.join(dim, "prod_id", "left").count())
```

Incorrect Output — Fix & Prevent

Q67 • Fix and verify

- **Correct the logic**, then test on the same input
- **Regression test** — run the full pipeline; downstream steps must not break
- **Historical data** — validate against known good outputs

Q68 • Prevent it coming back

Measure	Detail
Validation checks	Data-type, range and consistency checks inside the step
Output monitoring	Alert on anomalies in row counts and sums
Docs vs reality	Update documentation if it disagreed with the code
Share the fix	Communicate the issue and resolution to stakeholders
Post-mortem	Improve the process, not just the query

Q69 • Row count dropped 30% overnight?

Compare source counts first

Inner join on a null-heavy key?

A filter on a column whose casing or format changed?

Recent release or schema change?

★ Interview Tip

Scenario: **"the job succeeded but the numbers are wrong."** Most often it is join fan-out from a non-unique key. Say that, then say how you would prove it.

💡 Tip

Keep a **golden dataset** with known expected output. Every fix is proven against it before it ships.

Data Duplication

Q70 • Find the source

- Is it in the **source**, **extraction**, or a **transform**?
- Profile: count occurrences per business key
- Define **uniqueness** — single column or composite key

Q71 • Prevent at ingestion

- **Upsert** (MERGE) instead of blind insert
- **Staging area** — dedupe before production
- **Primary keys** / **UUIDs** for a stable identity

Q72 • Deduplicate in the pipeline

Tool	Technique
SQL	DISTINCT, or GROUP BY with aggregates
Spark	dropDuplicates(["key"]) — keeps an arbitrary row
Spark + window	row_number() over key, order by ts desc = 1 — keeps the latest
Delta	MERGE on the key — idempotent reloads

```
w = Window.partitionBy("cust_id").orderBy(F.desc("updated_at"))
clean = (df.withColumn("rn", F.row_number().over(w))
        .filter("rn = 1").drop("rn"))           # newest row wins
```

Q73 • Automate detection and governance

- Scheduled duplicate checks with a **threshold alert**
- Periodic audits + a user feedback loop for reports
- Governance: policies for handling duplicates, trained teams, data-quality tooling

★ Interview Tip

Plain `dropDuplicates(["id"])` keeps a random row. When recency matters, say **window + row_number** — and mention idempotent MERGE for re-delivered files.

💡 Tip

Dailyfile dropped twice? MERGE on the business key, or dedupe with a window. Either makes the load idempotent.

Errors & Logging in Distributed Jobs

Q74 • Classify errors

Type	Response
Transient	Retry with backoff
Permanent	Quarantine + fix data
Critical	Stop, alert, page on-call

Handle gracefully: skip bad records rather than failing the whole job.

Q75 • Structured logging

- **JSON** format — machine-parsable
- Include **job id, task id, timestamp**
- Add input params, message + stack trace, source metadata

Q76 • Central logging + alerts

Need	Tool / practice
Aggregate logs from every node	ELK, Splunk, CloudWatch, Stackdriver
Real-time dashboards	Spot error spikes and patterns quickly
Notifications	PagerDuty, Slack, email
Threshold alerts	Fire when the error rate crosses a limit

```
import json, logging
log = logging.getLogger("etl")
def log_err(job, task, e, rec=None):
    log.error(json.dumps({"job": job, "task": task,
        "error": type(e).__name__, "msg": str(e), "record": rec}))
```

Distributed Errors — Recovery

Q77 • Recover without starting over

Mechanism	Benefit
Checkpointing	Restart from the last saved state, not from zero
Compensating transactions	Undo or correct actions taken before the error
Pre-processing validation	Catch schema, type and range errors early
Post-processing checks	Confirm output meets quality standards
Maximum retry limits	No infinite loops or runaway cost

Q78 • Report, learn, share

- **Error report** after each run: types, frequency, likely causes
- **Root cause analysis** for significant errors
- **Runbooks + knowledge sharing** — document common issues and fixes
- **Review logs regularly**; feed lessons into the next job design

Q79 • Spark-specific tips

Problem	Fix
Task not serializable	Create connections inside mapPartitions
Bad records in files	PERMISSIVE mode + corrupt-record column
Lost state on restart	Checkpoint dir; keep it unique per query

★ Interview Tip

Say: "I classify errors, retry the transient ones, quarantine the bad data and alert on rates, not single failures." Then name your logging stack.

◆ Tip

Put a run id in every log line. One id lets you trace a failure across hundreds of executors in seconds.

Rapid-Fire Scenarios

Q80 • One-line first moves

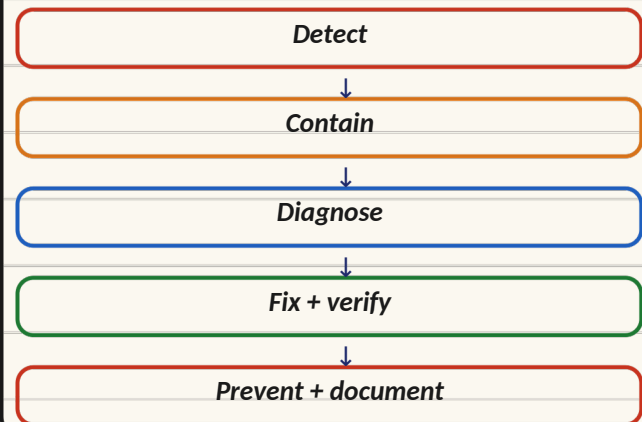
Scenario	First move
ETL job failed overnight	Read the error; transient? retry. Permanent? quarantine + alert
Pipeline suddenly slow	Time each stage; compare with history; check data volume
Bad data in a huge dataset	Profile, validate, quarantine — never silently drop
Schedule did not fire	Time zone + cron first, then dependencies, then scheduler logs
Spark executor OOM	Check partition size and skew before adding memory
Source column renamed	Explicit schema on read; alert on drift; evolve deliberately
Third-party API returns 429	Back off with jitter; respect rate limits; cache fallback
Two stores disagree	Reconcile counts + sums; find the source of truth; use CDC
Numbers wrong, job green	Suspect join fan-out from a non-unique key
File loaded twice	MERGE on the key or window dedupe — make loads idempotent

Q81 • Answer framework

For any "what would you do if..." question, walk the same five steps out loud. It sounds senior because it is **structured**, not because it is long.

Finish with a **real war story**: the symptom, what you saw, what you changed, and the before/after.

HOW IT FLOWS

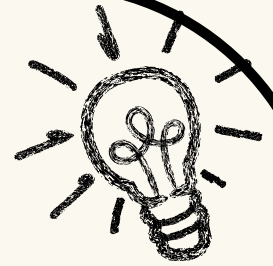


★ Interview Tip

Almost every operations question reduces to **what changed, where, and how do we stop it recurring**. Answer through that lens and you will sound senior.

You've covered 14 scenarios — now go break a pipeline and fix it. That is where the real answers come from.

“



Created by @Ganesh R

I hope you have found this **Document** actionable to
become a **DATA ENGINEER/ANALYST**

Join **with us** to get more updates
stuff Similar to this you finished reading



RG_Data_Talks(Azure, AWS)



RG_Data_Talks(Azure, AWS)

”

Thank you



If You Found This Helpful?



Like



Comment



Repost



Share



Save



Download

**Follow for more amazing content
like this !**